



Centre for Credible
Artificial Intelligence

Warsaw University of Technology

Piotr Suszyński

24 August 2026

Seminar: Verifying Chain-of-Thought Reasoning via Its Computational Graph

Published as a conference paper at ICLR 2026

VERIFYING CHAIN-OF-THOUGHT REASONING VIA ITS COMPUTATIONAL GRAPH

Zheng Zhao^{1,3*} Yeskendir Koishekenov¹ Xianjun Yang² Naila Murray¹ Nicola Cancedda¹

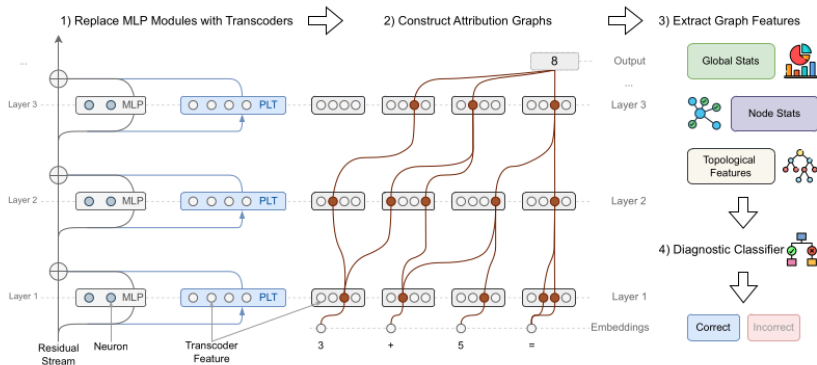
¹FAIR at Meta ²Meta Superintelligence Labs ³University of Edinburgh

zheng.zhao@ed.ac.uk, yeskendir@meta.com, xianjunyang@ucsb.edu
murrayn@meta.com, canceddan@acm.org

ABSTRACT

Current Chain-of-Thought (CoT) verification methods predict reasoning correctness based on outputs (black-box) or activations (gray-box), but offer limited insight into *why* a computation fails. We introduce a white-box method: **Circuit-based Reasoning Verification (CRV)**. We hypothesize that attribution graphs of correct CoT steps, viewed as *execution traces* of the model’s latent reasoning circuits, possess distinct structural fingerprints from those of incorrect steps. By training a classifier on structural features of these graphs, we show that these traces contain a powerful signal of reasoning errors. Our white-box approach yields novel scientific insights unattainable by other methods. (1) We demonstrate that structural signatures of error are highly predictive, establishing the viability

Quick summary of Zhao et al. ICLR 2026



(from Zhao et al. ICLR 2026)

Transcoders Find Interpretable LLM Feature Circuits

Jacob Dunefsky*
Yale University
New Haven, CT 06511
jacob.dunefsky@yale.edu

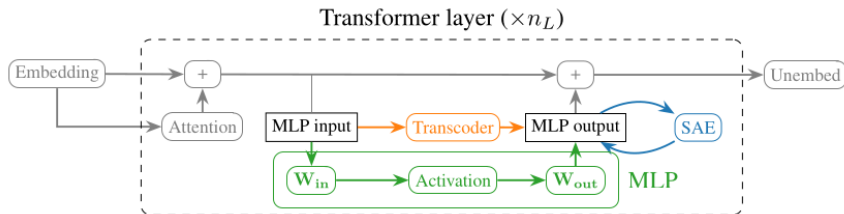
Philippe Chlenski*
Columbia University
New York, NY 10027
pac@cs.columbia.edu

Neel Nanda

Abstract

A key goal in mechanistic interpretability is circuit analysis: finding sparse sub-graphs of models corresponding to specific behaviors or capabilities. However, MLP sublayers make fine-grained circuit analysis on transformer-based language models difficult. In particular, interpretable features—such as those found by sparse autoencoders (SAEs)—are typically linear combinations of extremely many neurons, each with its own nonlinearity to account for. Circuit analysis in this setting thus either yields intractably large circuits or fails to disentangle local and global behavior. To address this we explore **transcoders**, which seek to faithfully approximate a densely activating MLP layer with a wider, sparsely-activating MLP layer. We introduce a novel method for using transcoders to perform weights-based circuit analysis through MLP sublayers. The resulting circuits neatly factorize into

MLP $\rightarrow$ transcoder



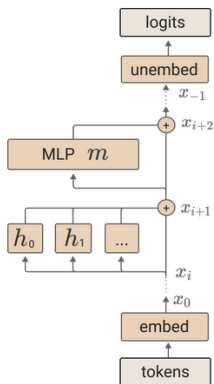
$$\mathbf{z}_{TC}(\mathbf{x}) = \text{ReLU}(\mathbf{W}_{\text{enc}}\mathbf{x} + \mathbf{b}_{\text{enc}})$$

$$\text{TC}(\mathbf{x}) = \mathbf{W}_{\text{dec}}\mathbf{z}_{TC}(\mathbf{x}) + \mathbf{b}_{\text{dec}},$$

$$\mathcal{L}_{TC}(\mathbf{x}) = \underbrace{\|\text{MLP}(\mathbf{x}) - \text{TC}(\mathbf{x})\|_2^2}_{\text{faithfulness loss}} + \lambda_1 \underbrace{\|\mathbf{z}_{TC}(\mathbf{x})\|_1}_{\text{sparsity penalty}}.$$

(from Dunevsky et al. NeurIPS 2024)

Transformer residual stream



The final logits are produced by applying the unembedding.

$$T(t) = W_U x_{-1}$$

An MLP layer, m , is run and added to the residual stream.

$$x_{i+2} = x_{i+1} + m(x_{i+1})$$

Each attention head, h , is run and added to the residual stream.

$$x_{i+1} = x_i + \sum_{h \in H_i} h(x_i)$$

One residual block

Token embedding.

$$x_0 = W_E t$$

(from "A Mathematical Framework for Transformer Circuits", Elhage et al., 2021, Anthropic)

Transformer block:

$$\begin{aligned}\mathbf{x}_{\text{mid}}^{(l,t)} &= \mathbf{x}_{\text{pre}}^{(l,t)} + \sum_{\text{head } h} \text{attn}^{(l,h)} \left(\mathbf{x}_{\text{pre}}^{(l,t)}; \mathbf{x}_{\text{pre}}^{(l,1:t)} \right) \\ \mathbf{x}_{\text{pre}}^{(l+1,t)} &= \mathbf{x}_{\text{mid}}^{(l,t)} + \text{MLP}^{(l)} \left(\mathbf{x}_{\text{mid}}^{(l,t)} \right)\end{aligned}$$

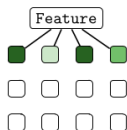
Attribution between transcoder feature pairs:

$$\underbrace{z_{TC}^{(l,i)} \left(\mathbf{x}_{\text{mid}}^{(l,t)} \right)}_{\text{input-dependent}} \underbrace{\left(\mathbf{f}_{\text{dec}}^{(l,i)} \cdot \mathbf{f}_{\text{enc}}^{(l',i')} \right)}_{\text{input-invariant}}$$

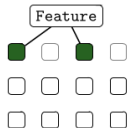
Attribution through attention heads:

$$\text{score}^{(l,h)} \left(\mathbf{x}_{\text{pre}}^{(l,t)}, \mathbf{x}_{\text{pre}}^{(l,s)} \right) \left(\left(\left(\mathbf{W}_{\text{OV}}^{(l,h)} \right)^T \mathbf{f}_{\text{enc}}^{(l',i')} \right) \cdot \mathbf{x}_{\text{pre}}^{(l,s)} \right)$$

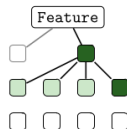
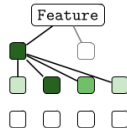
Building attribution graph



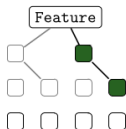
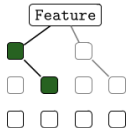
Step 1: find the direct contributions of earlier-layer features to a later-layer feature.



Step 2: delete all but the k most highly-contributing features.



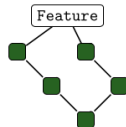
Step 3: select the top- k contributing features for each selected features.



Step 4: delete all but the k most important *paths* in the graph again.



Step 5: Repeat steps 2–4 until you have paths of length l .



Step 6: Merge top k paths into a single circuit.

(from Dunevsky et al. NeurIPS 2024)

- ▶ Transcoder as MLP replacement
- ▶ Transcoder vs SAE interpretability
- ▶ Circuit analysis case studies
 - ▶ tracing and analyzing individual features and circuits on a single example
 - ▶ "Polish" surnames: `oglu` , `owsky` , `zyk` , `chenko` and `kowski` as determined by US-based authors named Jacob Dunefsky and Philippe Chlenski

back to Zhao et al. ICLR 2026..

Extracting features from the graph

1. Global Graph Statistics: These features provide a high-level summary of the pruned computational graph.

- *Node Counts:* The total number of active transcoder features, as well as the count of transcoder feature nodes and residual stream nodes remaining after pruning. This captures the overall sparsity and composition of the influential subgraph.
- *Logit Statistics:* The probability of the top-ranked token and the entropy of the final logit distribution. These classic uncertainty measures serve as simple but informative baseline features.

2. Node Influence and Activation Statistics: This group of features characterizes the properties of the nodes within the pruned graph, moving beyond simple counts.

- *Influence Scores:* The mean influence of all nodes in the pruned graph, along with the total and mean influence specifically from the residual stream (“error”) nodes. This helps quantify how much of the final output is attributed to specific learned features versus the model’s direct pass-through states.
- *Activation Statistics:* For the pruned transcoder feature nodes, we compute the mean, max, and standard deviation of their activation values. This captures the intensity and distribution of the active, interpretable features. A high maximum activation, for instance, might signal that a single, highly decisive feature was responsible for the step.
- *Layer-wise Feature Histogram:* A histogram of active transcoder features across the model’s layers. This feature vector characterizes the distribution of computational effort across the model’s depth, allowing us to test hypotheses such as whether errors correlate with the activation of components at specific layers.

3. Topological and Path-Based Features: To capture the structure and efficiency of the information flow, we compute a rich set of topological features on the pruned, directed subgraph.

- *Edge and Density Statistics:* Aggregate statistics on the edge weights (sum, mean, std), the total number of edges, and the graph density. We hypothesize that a sparse, fragmented graph (low density, few edges) may indicate a breakdown in information flow characteristic of an error.
- *Centrality Measures:* To identify critical “hub” nodes in the computation, we calculate the mean and max for both degree centrality and weighted betweenness centrality. These features assess whether influence is concentrated or diffused.
- *Connectivity and Path Lengths:* The number of weakly connected components and the average shortest path length within the largest component. A highly fragmented graph may suggest a failed computation. A particularly crucial feature is the **shortest path length from any input token node to any final logit node**. This directly measures how efficiently information from the prompt propagates to the final decision. A long or non-existent path is hypothesized to be a strong signal that the model is “ignoring” its instructions or context.

(from Zhao et al. ICLR 2026)

- ▶ Three **custom** datasets: Boolean, Arithmetic, GSM8K
- ▶ LLM-as-a-judge (+ direct calculation) used to generate labels
- ▶ Each CoT step is a dataset entry

Arithmetic ($n = 7$)

Expression: $(((-3) + (-6)) * (9 * 6)) + (-4)$

Generated CoT: To evaluate this expression, we need to follow the order of operations (PEMDAS):

1. Evaluate the expressions inside the innermost parentheses:

$$* (-3) + (-6) = -9$$

2. Multiply 9 and 6:

$$* 9 * 6 = 54$$

3. Multiply -9 and 54:

$$* -9 * 54 = -486$$

4. Add -4 to -486:

$$* -486 + (-4) = -490$$

The final answer is -490.

(from Zhao et al. ICLR 2026)

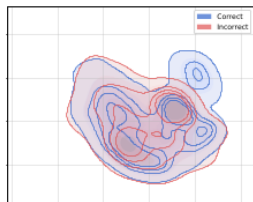
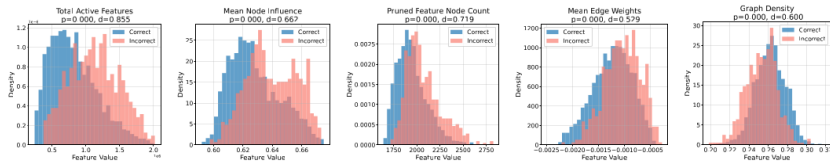
Paradigm	Method	Synthetic (Boolean)			Synthetic (Arithmetic)			GSM8K		
		AUROC $\uparrow$	AUPR $\uparrow$	FPR@95 $\downarrow$	AUROC $\uparrow$	AUPR $\uparrow$	FPR@95 $\downarrow$	AUROC $\uparrow$	AUPR $\uparrow$	FPR@95 $\downarrow$
Black-Box	MaxProb	<u>58.81</u>	0.34	95.20	61.87	1.81	84.98	54.91	7.99	91.86
	PPL	<u>57.37</u>	0.29	91.02	60.19	1.68	85.52	55.46	8.12	90.69
	Entropy	53.56	0.24	97.55	60.03	1.52	85.40	56.67	7.29	87.08
	Temp. Scaling	58.77	0.36	91.41	59.67	1.66	86.96	54.42	8.24	92.28
	Energy	51.08	0.28	95.11	<u>76.45</u>	<u>5.59</u>	73.86	<u>62.55</u>	9.11	<u>86.34</u>
Gray-Box	CoE-R	53.17	0.33	92.85	58.47	1.93	76.68	52.38	8.34	96.20
	CoE-C	51.03	<u>0.38</u>	92.07	69.39	3.03	<u>63.33</u>	53.57	<u>10.80</u>	96.33
	CoT-Kinetics	53.62	0.24	97.13	60.83	1.58	85.09	56.54	7.35	86.83
	LR Probe	52.91	0.25	<u>88.42</u>	54.22	1.50	91.90	55.86	7.99	90.32
	MLP Probe	53.63	0.26	<u>88.56</u>	54.41	1.30	90.98	56.02	8.63	93.94
White-Box	CRV (Ours)	75.87	0.97	79.17	92.47	28.92	37.09	70.17	14.3	79.61

(from Zhao et al. ICLR 2026)

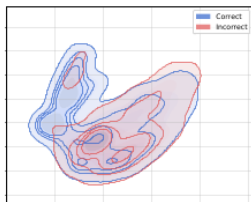
Cross-domain generalization

Test Set	Method (Train Set)	Metrics		
		AUROC $\uparrow$	AUPR $\uparrow$	FPR@95 $\downarrow$
Boolean	Baseline (MaxProb)	58.81	0.34	95.20
	CRV (GSM8K)	45.77	0.21	97.28
	CRV (Arithmetic)	61.58	0.51	87.55
	CRV (Combined)	65.18	0.65	80.15
	<i>CRV (Boolean)</i>	<i>75.87</i>	<i>0.97</i>	<i>79.17</i>
Arithmetic	Baseline (Energy)	76.45	5.59	73.86
	CRV (GSM8K)	55.11	1.50	91.91
	CRV (Boolean)	69.59	2.64	72.87
	CRV (Combined)	90.51	30.57	44.23
	<i>CRV (Arithmetic)</i>	<i>92.47</i>	<i>28.92</i>	<i>37.09</i>
GSM8K	Baseline (Energy)	62.55	9.11	86.34
	CRV (Boolean)	44.37	6.33	95.71
	CRV (Arithmetic)	57.04	7.85	94.37
	CRV (Combined)	70.62	12.66	80.22
	<i>CRV (GSM8K)</i>	<i>70.17</i>	<i>14.3</i>	<i>79.61</i>

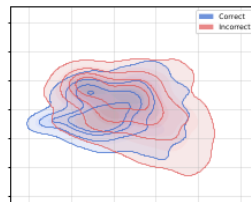
(from Zhao et al. ICLR 2026)



(a) Boolean



(b) Arithmetic



(c) GSM8K

(from Zhao et al. ICLR 2026)

- ▶ manual circuit analysis
- ▶ finding an "unusually high activation" feature and determining that it is a "multiplication" feature (not explained how)
- ▶ setting the feature to zero

Before Intervention (Incorrect)	After Intervention (Correct)
Evaluate the arithmetic expression below. $(7 * ((5 + 9) + 7))$ To evaluate this expression, we need to follow the order of operations (PEMDAS): 1. Evaluate the expression inside the innermost parentheses: $5 + 9 = 14$ 2. Multiply 7 by the result: $7 * 14 = 98$ 3. Add 7 to the result: $98 + 7 = 105$ Therefore, the expression evaluates to 105.	Evaluate the arithmetic expression below. $(7 * ((5 + 9) + 7))$ To evaluate this expression, we need to follow the order of operations (PEMDAS): 1. Evaluate the expression inside the innermost parentheses: $5 + 9 = 14$ 2. Add 7 to the result: $14 + 7 = 21$ 3. Multiply 7 by the result: $7 * 21 = 147$ Therefore, the value of the expression is 147.

(from Zhao et al. ICLR 2026)

- ▶ vastly improved metrics indicate there is correct/incorrect signal in the graph structure
- ▶ finding that error signatures are domain specific (and this was not a MoE model)
- ▶ showcase of a possible intervention, validating claim that the method improves interpretability

- ▶ expensive computationally
- ▶ they claim to be "white box", yet they use black-box features
- ▶ the efficacy metrics are still very bad, the AUPR of 0.97 is not 97%, it really is 0.97%. For GSM8K or Boolean the FPR@95 equal to $\sim 79\%$ - this actually raises question why comparator methods are *even* worse
- ▶ causal intervention completely manual and it is not specified how exactly authors determined the feature is "multiplication"
- ▶ lack of standard deviations anywhere

Weaknesses - no standard deviations comparison

Table 14 (ablation for training on instruction-tuning data):

Transcoder Training	Synthetic (Boolean)			Synthetic (Arithmetic)			GSM8K		
	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓
Base	75.87	0.97	79.17	92.47	28.92	37.09	70.17	14.3	79.61
+ IT Data	76.04	1.20	66.82	91.39	28.44	38.47	72.01	15.40	83.27

"did not yield a consistent or meaningful improvement" (5 better, 4 worse)

Table 15 (ablation for attribution position):

Attribution Position	Synthetic (Boolean)			Synthetic (Arithmetic)			GSM8K		
	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓
Before	68.66	1.80	77.44	85.95	12.05	47.89	70.32	16.19	85.29
After	75.87	0.97	79.17	92.47	28.92	37.09	70.17	14.3	79.61

"show a clear and consistent advantage" (5 better, 4 worse)

(from Zhao et al. ICLR 2026)

Weaknesses - non-linear model needed?

Authors say that PCA/t-SNE confirm the model learns "*complex boundary*" and that "*ability to capture non-linear feature interactions is critical*", yet the model ablation shows Logistic Regression as competitive:

Method	Synthetic (Boolean)			Synthetic (Arithmetic)			GSM8K		
	AUROC $\uparrow$	AUPR $\uparrow$	FPR@95 $\downarrow$	AUROC $\uparrow$	AUPR $\uparrow$	FPR@95 $\downarrow$	AUROC $\uparrow$	AUPR $\uparrow$	FPR@95 $\downarrow$
Dummy	50.8	0.25	100	49.84	1.20	100	48.06	6.46	100
Logistic Regression	76.4	0.75	68.91	89.5	11.46	41.56	73.8	18.70	78.69
Random Forest	61.71	4.49	100	92.99	43.68	30.56	71.7	17.65	76.18
Gradient Boosting	75.87	0.97	79.17	92.47	28.92	37.09	70.17	14.3	79.61

(from Zhao et al. ICLR 2026)

Weaknesses - are the results trustworthy?

- ▶ all datasets were created by authors (also true for GSM8K - labels built by authors)
- ▶ comparator methods hyperparameter optimization is bad - best values at the end of search range, small search space
- ▶ extremely bad performance of comparator methods - are these really state of the art?
- ▶ validation by humans (authors) of LLM-as-a-judge – failed to do arithmetic to justify LLM use?

Comparison	Cohen's Kappa (κ)	Agreement (%)
Human vs. Human (Inter-Annotator)	0.42	87.3%
Human vs. LLM-as-a-Judge	0.26	84.1%

(from Zhao et al. ICLR 2026)



Centre for Credible
Artificial Intelligence

Warsaw University of Technology

Thank you!